

Wireshark Capture & Display Filters

1. Introduction to Wireshark

Wireshark is an open-source, cross-platform network packet analyzer used to capture, inspect, and analyze network traffic in real time or from packet capture (PCAP/PCAPNG) files. It enables deep inspection of protocols across all OSI layers and is widely used in networking, security analysis, troubleshooting, digital forensics, and education.

Core Use Cases

- Network troubleshooting (latency, packet loss, congestion)
- Security analysis (anomalous traffic, scans, MITM, malware indicators)
- Protocol learning and validation
- Incident response and forensic investigation

Wireshark is not an IDS/IPS. It does not block or modify traffic; effectiveness depends on analyst skill.

2. Wireshark GUI Architecture

Wireshark uses a unified interface composed of several functional areas:

Main Components

- **Toolbar** – Capture control, file operations, filtering, statistics
- **Display Filter Bar** – Query-based packet filtering
- **Recent Files** – Quick access to previous captures
- **Capture Interfaces & Filters** – Interface selection and capture filters
- **Status Bar** – Capture state, profile, packet counters

Packet Analysis Panes

1. **Packet List Pane** – Summary view (No., Time, Source, Destination, Protocol, Length, Info)
 2. **Packet Details Pane** – Protocol tree view (layer-by-layer decoding)
 3. **Packet Bytes Pane** – Hexadecimal and ASCII representation
-

3. Working with Capture Files

Loading PCAP Files

- File → Open
- Drag and drop
- Double-click a file

Merging Capture Files

- File → Merge
- Combines multiple captures into a single PCAP

Viewing Capture Metadata

- Statistics → Capture File Properties
 - Includes hashes, timestamps, interfaces, comments, and statistics
-

4. Packet Dissection & OSI Model Mapping

Wireshark decodes packets according to OSI layers:

1. **Frame (Layer 1)** – Capture metadata, timestamps, frame length
2. **Data Link (Layer 2)** – MAC addresses, Ethernet headers
3. **Network (Layer 3)** – IP addressing, TTL, fragmentation
4. **Transport (Layer 4)** – TCP/UDP ports, flags, sequencing
5. **Session/Presentation (Layer 5–6)** – Reassembly, protocol state
6. **Application (Layer 7)** – HTTP, DNS, FTP, SMB, etc.

Each selected field highlights corresponding bytes, aiding forensic validation.

5. Packet Navigation & Annotation

Packet Numbering

- Unique sequential ID per packet
- Enables precise navigation and correlation

Searching Packets

- Edit → Find Packet
- Input types: Display filter, Hex, String, Regex
- Search scope: Packet list, details, or bytes

Marking & Commenting

- Mark packets for emphasis (session-based)
 - Add comments stored within the capture file
-

6. Time & Visualization Features

Time Display Formats

- Seconds since capture start (default)
- Absolute UTC / Local time
- Delta times between packets

Packet Coloring

- Default protocol-based coloring rules
 - Custom rules using display filters
 - Temporary vs permanent rules
-

7. Expert Information System

Wireshark flags protocol anomalies and noteworthy events.

Severity Levels

- **Chat (Blue)** – Normal behavior

- **Note (Cyan)** – Informational events
- **Warn (Yellow)** – Suspicious or unusual
- **Error (Red)** – Malformed or problematic packets

Categories include checksum errors, deprecated usage, malformed packets, and comments.

8. Packet Filtering Concepts

Filter Types

- **Capture Filters** – Applied before capture (BPF syntax)
- **Display Filters** – Applied after capture (Wireshark syntax)

Filtering Philosophy

| Capture everything, filter during analysis.

9. Display Filter Syntax & Operators

Comparison Operators

- `==` Equal
- `!=` Not equal
- `>` `<` `>=` `<=`

Logical Operators

- `and` / `&&`
- `or` / `||`
- `not` / `!`

Advanced Operators & Functions

- `contains` – Substring match (case-sensitive)
- `matches` – Regex matching
- `in {}` – Set membership

- `upper()` / `lower()` – Case conversion
 - `string()` – Type conversion
-

10. Right-Click Analysis Features

- Apply as Filter
 - Prepare as Filter
 - Conversation Filter
 - Colorize Conversation
 - Apply as Column
-

11. Stream Reconstruction

- Follow TCP / UDP / HTTP streams
 - Reassembles application-layer data
 - Reveals cleartext credentials and payloads (unencrypted traffic)
-

12. Statistical & Traffic Overview Tools

Key Statistics Menus

- Protocol Hierarchy
- Conversations
- Endpoints
- Resolved Addresses
- IPv4 / IPv6 Statistics
- DNS Statistics
- HTTP Statistics

Supports name resolution and optional GeoIP mapping.

13. Security Analysis Scenarios

Nmap Scan Detection

- **TCP Connect Scans** – Full handshake
- **SYN Scans** – Half-open connections
- **UDP Scans** – ICMP Port Unreachable responses

ARP Poisoning / MITM Detection

Indicators:

- Duplicate IP-MAC mappings
- Gratuitous ARP floods
- Gateway impersonation

Host Identification

Protocols used:

- DHCP (hostnames, leases)
- NetBIOS (device names)
- Kerberos (user & service authentication)

Tunneling Detection

- ICMP tunneling (large payloads)
- DNS tunneling (long, encoded subdomains)

Cleartext Protocol Risks

- FTP credential exposure
 - Data exfiltration
 - Malware delivery
-

Wireshark Cheat Sheet

Capture Filters

```
tcp port 80  
udp port 53  
host 192.168.1.1  
net 10.0.0.0/24
```

Display Filters – Core

```
ip  
tcp  
udp  
dns  
http
```

IP Filters

```
ip.addr == 10.10.10.111  
ip.src == 10.10.10.111  
ip.dst == 10.10.10.111
```

TCP / UDP Filters

```
tcp.port == 80  
udp.port == 53  
tcp.srcport == 1234  
tcp.dstport == 443
```

HTTP Filters

```
http  
http.request.method == "GET"  
http.response.code == 200  
http.server contains "Apache"
```

DNS Filters

```
dns
dns.flags.response == 0
dns.qry.type == 1
dns.qry.name.len > 15
```

TCP Flag Filters (Scanning)

```
tcp.flags.syn == 1
tcp.flags.ack == 1
tcp.flags.reset == 1
tcp.flags == 18
```

ARP Filters

```
arp
arp.opcode == 1
arp.opcode == 2
arp.duplicate-address-detected
```

DHCP Filters

```
dhcp
dhcp.option.hostname contains "host"
dhcp.option.dhcp == 3
```

Kerberos Filters

```
kerberos
kerberos.CNameString contains "user"
!(kerberos.CNameString contains "$" )
```

ICMP & Tunneling


```
icmp  
data.len > 64 and icmp
```

Advanced Functions

```
contains  
matches  
in {80 443 8080}  
upper(field)  
lower(field)  
string(field)
```

14. Real-World Lab Examples (Hands-On Scenarios)

This section provides practical, real-world lab-style scenarios that mirror how Wireshark is used in professional environments such as SOC's, enterprise networks, and incident response investigations. Each lab includes an objective, scenario, filters used, and analyst outcome.

Lab 1: Detecting HTTP Cleartext Credential Exposure

Objective: Identify usernames and passwords transmitted over HTTP.

Scenario: A legacy web application is suspected of transmitting login credentials without encryption.

Steps:

1. Open the capture file in Wireshark.
2. Apply the filter:

```
http
```

3. Right-click any HTTP packet → Follow → TCP Stream.
4. Inspect POST requests.

Key Filters Used:

```
http.request.method == "POST"  
http contains "username"
```

Outcome: Credentials are visible in cleartext, confirming insecure authentication.

Lab 2: Identifying a TCP SYN Port Scan (Nmap -sS)

Objective: Detect stealth scanning activity.

Scenario: IDS alerts indicate possible reconnaissance from an internal host.

Steps:

1. Apply SYN-only filter:

```
tcp.flags.syn == 1 and tcp.flags.ack == 0
```

2. Observe repeated SYN packets to multiple destination ports.

Indicators:

- No handshake completion
- Small TCP window size

Outcome: Activity matches Nmap SYN scan behavior.

Lab 3: Detecting TCP Connect Scan (Nmap -sT)

Objective: Identify full TCP handshake scans.

Scenario: A non-privileged user attempts network mapping.

Filter Used:

```
tcp.flags.syn==1 and tcp.flags.ack==0 and tcp.window_size > 1024
```

Outcome: Full three-way handshakes confirm TCP Connect scan.

Lab 4: ARP Poisoning / MITM Detection

Objective: Identify Man-in-the-Middle attack via ARP spoofing.

Scenario: Users report intermittent connectivity issues.

Steps:

1. Apply ARP filter:

```
arp
```

2. Check for duplicate IP-to-MAC mappings.
3. Open Expert Information.

Advanced Filter:

```
arp.duplicate-address-detected
```

Outcome: One MAC address impersonates the gateway → confirmed MITM.

Lab 5: DHCP Host Identification

Objective: Identify hostnames and IP allocation.

Scenario: Unknown host detected on the network.

Filters Used:

```
dhcp.option.dhcp == 3  
dhcp.option.hostname
```

Outcome: Hostname and requested IP identified from DHCP Request packets.

Lab 6: DNS Tunneling Detection

Objective: Detect data exfiltration via DNS.

Scenario: Malware suspected of using DNS for C2 communication.

Filters Used:

```
dns.qry.name.len > 15 and !mdns
```

Indicators:

- Long, encoded subdomains
- Repetitive queries to same domain

Outcome: DNS tunneling behavior identified.

Lab 7: ICMP Tunneling Detection

Objective: Detect covert data transfer over ICMP.

Scenario: Sudden spike in ICMP traffic observed.

Filter Used:

```
data.len > 64 and icmp
```

Outcome: ICMP packets carrying excessive payloads indicate tunneling.

Lab 8: FTP Credential Theft Analysis

Objective: Extract FTP credentials from network traffic.

Scenario: Legacy FTP server compromise suspected.

Filters Used:

```
ftp
ftp.request.command == "USER"
ftp.request.command == "PASS"
```

Steps:

- Follow TCP Stream
- Extract username and password

Outcome: Credentials recovered in cleartext.

Lab 9: Suspicious HTTP Download (Malware Delivery)

Objective: Detect malicious file download.

Scenario: Endpoint alerts indicate malware execution.

Filters Used:

```
http.response.code == 200
http contains ".exe"
```

Steps:

- File → Export Objects → HTTP

Outcome: Executable extracted for malware analysis.

Lab 10: Identifying Kerberos User Accounts

Objective: Identify authenticated users in a Windows domain.

Scenario: Investigating lateral movement.

Filters Used:

```
kerberos.CNameString and !(kerberos.CNameString contains "$" )
```

Outcome: Valid domain usernames identified.

Conclusion

This report now includes both **theoretical mastery and hands-on practical labs**, enabling readers to not only understand Wireshark concepts but also apply them in real-world security operations, incident response, and forensic investigations.